Contents

The rgtlab Style: A Workflow Reference Derived from the ryyblog Corpus **1**
Abstract . . . 1
1. Scope, evidence, and epistemic status . . . 2
2. The organizing idea: a layered reference architecture . . . 3
3. Configuration: dotfiles as source code . . . 3
4. File system and naming conventions . . . 4
5. The shell layer . . . 5
5.1 zsh configuration . . . 5
5.2 Modern CLI replacements . . . 5
5.3 The scripts-versus-functions rule . . . 6
6. Version control . . . 7
7. Security . . . 7
7.1 The three-tier secrets scheme . . . 7
7.2 The threat model and its controls . . . 8
8. Sync and storage . . . 9
8.1 The three-layer framing . . . 9
8.2 The migration discipline . . . 10
8.3 The reversal: object storage over sync . . . 10
8.4 The five-layer architecture at five machines . . . 10
9. Backup . . . 11
10. Reproducibility . . . 12
10.1 The two-tier stack . . . 12
10.2 The five pillars . . . 12
10.3 Testing . . . 12
10.4 Tiered CI . . . 13
11. Authoring and publishing . . . 13
11.1 Quarto as the document layer . . . 13
11.2 The post template . . . 13
11.3 House style . . . 14
11.4 Review methodology . . . 14
12. AI-augmented work . . . 15
13. The style, stated as invariants . . . 15
14. Internal tensions and open items . . . 16
15. What the corpus does not cover . . . 17
Appendix: post index by cluster . . . 17

The rgtlab Style: A Workflow Reference Derived from the ryyblog Corpus

2026-09-12 11:41 PDT

Abstract

The ryyblog corpus documents, across sixty-one posts in twelve thematic clusters, a single personal research-computing practice sustained over several years and several machines. No post in the

corpus is a statement of that practice as a whole; the closest is 'A Workflow Construct for the Modern Data Scientist', which names thirteen layers but defers the detail of each to a companion post. This paper reads the corpus as a whole and extracts the style it collectively encodes: the configuration doctrine, the Unix tool selection and the reasoning behind it, the security and secrets architecture, the sync and storage position, the reproducibility stack, the authoring conventions, and the terms on which generative-AI assistance is admitted into the workflow.

The practice is unusually self-documenting for a personal setup, and it is coherent enough to be stated as a small set of invariants. It is also, in several places, self-correcting: later posts openly reverse earlier recommendations, and one post measures the author's own machine and reports that a layer earlier described as complete is not. Those corrections are part of the style and are reported here as such.

---

1. Scope, evidence, and epistemic status

Corpus. 61 posts with `index.qmd` sources under `~/prj/ryyblog/posts/`, totalling approximately 289,000 words, organized into twelve declared clusters:

Cluster	Posts	Subject
`zzcollab-compendia`	9	Reproducible project scaffolding, CI
`penguins-arc`	8	Worked statistical analysis series
`workflow-construct`	7	Hardware through packages, layer by layer
`r-packages`	7	Package development, testing, review
`publishing`	7	Quarto, textbooks, video, deployment
`clinical-and-cloud`	4	EDC, AWS, Shiny hosting
`workflow-extensions`	3	LLM-augmented editing and adjacents
`security-and-sync`	3	Backup, `pass`, threat model
`cloud-sync-migration`	3	Dropbox exit, object storage
`shiny`	2	Prototyping, Observable interop
`shell-and-git`	2	Scripts versus functions, `zzgit`
`r-language`	2	Metaprogramming, pipe semantics

Method and its limits. Every post's YAML front matter and full heading tree was read. Ten posts were read in full: `wf-construct-overview-anchor`, `wf-unix-workspace-config`, `sh-scripts-vs-functions`, `sec-multi-laptop-threat-model`, `sec-aws-and-pass-secrets`, `sec-dropbox-to-portable-sync`, `sec-five-machine-storage`, `wf-claude-code-in-shell`, `zc-quarto-compendium-intro`, and (in part) `wf-modern-cli-tools`. Substantial extracts were read from `zc-tiered-ci-strategy`, `rp-code-review-methodology`, and `sec-three-tier-backup-architecture`. The remaining posts contribute through their headings, descriptions, and cross-references only.

Claims below are marked where the evidence is thinner than the statement: **(read)** for a claim taken from a post read in full, **(heading)** for a claim inferred from a post's structure and description without reading its body. Nothing in this paper was verified by running the described configuration; this is a document review, not an audit of the live machines.

---

2. The organizing idea: a layered reference architecture

The corpus does not treat a research workstation as a collection of installed tools. It treats it as a **layered reference architecture** in the sense of Bass, Clements and Kazman, with thirteen named layers, each occupied by a tool that is explicitly declared replaceable while the layer is not.

The declared layers, with their canonical occupants:

Layer	Occupant
Hardware	MacBook plus ThinkPad (macOS and Linux surface area)
Operating system	macOS (Sequoia), Linux Mint 22
File system	`~/prj` as the single canonical project root
Shell	zsh 5.9
Editor	Vim and Neovim, identical config locally and over SSH
Editor plugins	`zzvim-r`, `ultisnips`, `vimtex`
Shell functions	Only those that must mutate the calling shell
Shell scripts	`zzcollab`, `tn`, `zzgit`, `lssince`
Aliases	Two-letter shortcuts for high-frequency operations
Applications	`gh`, R, RStudio, Docker
Cloud	AWS EC2 plus a dotfiles repository
Backup	GitHub plus external SSD (plus Time Machine)
R packages	Author-maintained `zz*` namespace

Four framings are named explicitly in the corpus as the formal counterparts of what the construct does informally: Infrastructure as Code applied at the workstation level (with Nix Home Manager named as the rigorous alternative that was not adopted because it resists incremental migration); the reproducible research compendium of Gentleman and Temple Lang, as operationalized by Marwick, Boettiger and Mullen; the Internal Developer Platform and Golden Path of platform engineering, which the construct is said to be structurally identical to at single-user scale; and the `devcontainer.json` and SPDX/CycloneDX specifications as the schema-validated versions of the editor and version-matrix rows. **(read)**

Two consequences of the layering are load-bearing throughout the rest of the corpus:

1. **Substitution is surgical.** Replacing `autojump` with `zoxide`, `tmux` with `zellij`, or AWS with Hetzner is presented as a change confined to one row. The corpus applies this literally: the zoxide migration is configured as `eval "$(zoxide init zsh --cmd j)"` specifically so the `j` muscle memory survives the substitution.
2. **The convention outlives the tool.** `~/prj/NN-name/` is stated to have outlasted three editors and two cloud providers. Naming the convention rather than the tool is identified as the source of that durability.

---

3. Configuration: dotfiles as source code

The governing metaphor is explicit: a dotfiles repository is the **source code for the workspace**, and it should live in git, be tested before deployment, and have a migration path for breaking changes.

Structure and deployment. Files are stored without leading dots (`zshrc`, not `.zshrc`) in `~/dotfiles/`; an `install.sh` adds the dot prefix when creating symlinks into `$HOME`. The installer is idempotent by way of a `link_file` pattern that backs up any existing target to `<file>.backup.TIMESTAMP` before symlinking, and it carries a `--dry-run` flag. A `Makefile` wraps it: `make test` (dry-run plus `shellcheck` lint), `make install`, `make update` (pull and re-install). **(read)**

Rules stated as non-negotiable:

- **`.gitignore` before `git init`.** Stated as a prerequisite rather than a best practice, on the grounds that a single committed credential requires a full `git filter-repo` rewrite plus rotation of everything exposed, and must be treated as compromised regardless of how briefly it was visible.
- **The dotfiles repository is private.** Even a credential-free `.gitconfig` discloses tooling, organizational paths, and workflow patterns.
- **Per-file symlinks into `~/bin`, never a top-level directory symlink.** A `~/bin -> ~/dotfiles/bin` link works on the machine that created it and dangles silently elsewhere.
- **Do not track anything a tool regenerates.** The stated heuristic: if a directory is regenerated by running a tool, it is not user configuration. Editor plugin directories are the canonical example; only the plugin manifest is tracked.
- **The second bootstrap is the test.** Setting up the primary machine is configuration; setting up the second machine is testing. Assumptions invisible on the first surface immediately on the second, and the corpus insists on doing this before the third machine is needed.

Portability mechanics. A single `.zshenv` exports `DOTFILES` and `PRJ_ROOT` with fallback defaults, and everything downstream references the variables rather than literal paths. Optional tools are wired behind `command -v tool >/dev/null &&` guards so the same `.zshrc` deploys cleanly to a machine where they are not yet installed. OS divergence is handled inside one file with `case "$OSTYPE" in darwin*) ... linux*) ... esac` rather than by forking the repository. macOS `launchd` plists are templated with a `__USER__` placeholder that `install.sh` substitutes at deploy time; the substituted copy in `~/Library/LaunchAgents/` is explicitly not the file to edit. **(read)**

Honest status. `sec-five-machine-storage` reports a measurement of the author's own primary Mac: `~/dotfiles` is a symlink into the Dropbox tree, with no `.git` and no `install.sh`. The post states plainly that an earlier post in the same cluster describes this layer as complete and that on that machine it is not. The pattern of publishing the measured gap rather than the intended state recurs and should be read as part of the style. **(read)**

---

4. File system and naming conventions

- **`~/prj/` is the single canonical project root.** All automation is permitted to assume this path. It is named as the convention that pays the most compounded interest.
- **`~/prj/NN-name/`**, a numbered lead-in directory containing the actual git repository under a name without the numeric prefix. The prefix orders projects chronologically in file browsers without polluting repository names. The lead-in directory is not itself under git and holds an `archive/` subdirectory for superseded drafts. The same pattern governs `~/prj/sfw/07-zzcollab/zzcollab/` and the blog posts.
- **XDG base directories** are the target for relocations. The worked Maildir example moves `~/Mail/` to `~/.local/share/mail/`, and the corpus links the XDG specification as the deciding

authority for `~/.cache` versus `~/.local/share` versus `~/.local/state`.

- **Relative symlinks only.** An absolute symlink breaks on any other machine; relative symlinks survive cloning, moving, and CI runners.
- **Raw data is immutable.** `analysis/data/raw_data/` is read-only by convention; everything generated goes to `derived_data/`, and each dataset carries a `README.md` recording source, date, and use restrictions.

---

5. The shell layer

5.1 zsh configuration

`.zshenv` holds only what non-interactive shells and background jobs would break without: `DOTFILES`, `PRJ_ROOT`, `EDITOR`, `VISUAL`. Everything interactive lives in `.zshrc`.

History is configured large and shared: `HISTSIZE` and `SAVEHIST` at 100,000, with `SHARE_HISTORY`, `HIST_IGNORE_DUPS`, `INC_APPEND_HISTORY`, and `HIST_VERIFY`. The stated rationale is that research commands (training runs, pipeline executions, database queries) are worth recalling weeks later, and that incremental append means a crashed terminal loses nothing. The corpus also notes the cost: `SHARE_HISTORY` interleaves history across concurrent sessions, and dropping back to `INC_APPEND_HISTORY` alone restores session-boundary semantics.

Navigation is by `cdpath=($PRJ_ROOT $PRJ_ROOT/.. $HOME)` plus `setopt auto_cd`, so a project name alone is a valid command. The prompt carries the git branch via `vcs_info`, justified as prevention for the most common research-repository error: committing to the wrong branch after switching experiments.

`.inputrc` is treated as a first-class configuration file rather than an afterthought, because R, the Python REPL, `psql`, and most database clients use GNU Readline rather than zsh's line editor. Setting `editing-mode vi` plus cursor-shape indicators (`vi-ins-mode-string`, `vi-cmd-mode-string`) removes the mode-switching cost of moving between zsh and any Readline client. **(read)**

5.2 Modern CLI replacements

Eight Rust-implemented utilities are adopted as a Shell-layer extension, explicitly **alongside** rather than instead of the historical tools, because scripts and remote sessions still need `grep`, `find`, `cat`, `ls`, and `cd`:

Replaces	Tool	Stated reason
`grep`	`ripgrep`	Recursive by default, respects `.gitignore`
`find`	`fd`	Shorter syntax, parallel, `.gitignore`-aware
`cat`	`bat`	Syntax highlighting, paging
`ls`	`eza`	Git-aware, tree mode
`cd`	`zoxide`	Frecency ranking, fuzzy match
selection	`fzf`	Interactive selection across history and files
`git diff`	`delta`	Side-by-side, syntax-highlighted
git porcelain	`lazygit`	TUI for review-heavy work

Two positions in this section characterize the style more generally. First, the gain claimed is **per-keystroke, not per-second**: better defaults, not raw speed, are what accumulates. Second, the migration is judged by whether it preserves existing muscle memory; `zoxide init zsh --cmd j` is called the single most consequential change precisely because it changes the database and not the habit. **(read)**

A small set of `fzf`-driven functions completes the layer: `ff` (find any file, `cd` to its directory), `rr` (find an `.R`/`.Rmd`/`.qmd` file, open in `$EDITOR`), `pp` (find and open a PDF). Each is legitimately a function because it must change the calling shell or its environment.

5.3 The scripts-versus-functions rule

The corpus states one rule and derives an entire refactoring programme from it:

> A helper should be a shell function only when it must mutate the calling shell. Otherwise it should be a versioned, executable script in `~/bin`.

The justification is mechanical rather than aesthetic. A function runs in the current shell process and can therefore change the working directory, `export` variables, redefine aliases and options, and manipulate the directory stack, history, or job table. A script runs in a `fork`/`execve` child and returns exactly three things across the boundary: `stdout`, `stderr`, and an exit code. Anything outside that four-item list of state mutations does not need the function form.

The five costs of putting a script's logic in a function are enumerated and mapped onto practices the R-trained reader already accepts: startup latency (a long `.zshrc` function is reparsed on every shell start, the same shape of cost as `library()` calls in `.Rprofile`); lint coverage (`shellcheck` reads scripts and does not introspect dotfile functions); version-control granularity (a script is a file with its own history and meaningful blame); testability (a script can be invoked with controlled arguments and a captured exit code); and reuse across contexts (`~/bin` is reachable by `cron`, `launchd`, Make, Docker, HPC submission scripts, and CI runners, while a function is reachable only from an interactive zsh session that sourced it).

The R analogy is stated directly: a shell function corresponds to a function loaded into the current R session, and a shell script corresponds to `Rscript` invoked from the command line. No experienced R user would put a batch pipeline in `.Rprofile`; the shell community is described as having been less attentive to the same boundary.

The triage matrix has five fates: keep as function (must `cd`, `export`, or modify aliases or history); promote function to script; demote script to alias (a single command line with no argument parsing); keep as script and harden; decommission. The first row is reported to hold four to twenty helpers in every toolbox examined; the rest absorb everything else.

Hardening, when a script is kept, means: a shebang matching the syntax actually used (zsh-only constructs such as `${(f)var}` require `#!/usr/bin/env zsh`, and a bash shebang fails unhelpfully), `set -euo pipefail` unless a documented reason says otherwise, `$(...)` instead of backticks, quoted expansions, a clean `shellcheck` pass with per-line `disable` comments that name a reason, a `-h`/`--help` flag above thirty lines, and no `.sh` extension on executables (reserved for files that are sourced). A `make lint` target plus a `shellcheck` pre-commit hook scoped to `~/bin/*` is the guardrail against regression. **(read)**

---

6. Version control

- **`main` as the default branch**, `nvim` as the git editor, `delta` as the pager, `diff3` conflict style, `colorMoved` on, and a small alias set (`st`, `lg`, `co`, `br`).
- **Authentication via `gh auth git-credential`.** The credential helper delegates all GitHub authentication to the GitHub CLI, so a single `gh auth login` per machine removes the need for personal access tokens or SSH passphrases in ordinary `git` operations. The helper path differs by OS (`/opt/homebrew/bin/gh` versus `/usr/bin/gh`), and `$(which gh)` is offered as the portable form.
- **Commit messages in the imperative mood**, explaining what and why rather than how, with prefixed conventions offered as optional. **(heading)**
- **`zzgit` as the staging gate.** An interactive git wizard that wraps `gitleaks` at staging time. It is referenced repeatedly across clusters as the mechanism by which every diff, whether hand-typed or produced by an assistant, passes a secret scan before reaching a remote. A `pre-commit` hook is recommended for the residual case of a commit made through an IDE or a script that bypasses `zzgit`.
- **One repository per unit of work.** Each blog post is its own public GitHub repository, justified by independent history, isolated CI, focused collaboration, and independent dependency pinning. The same logic produces 400-plus private repositories and, in turn, a bulk archival procedure.
- **Never a git working tree inside a cloud-mounted path.** This is stated as an absolute in at least three posts, with the mechanism named: git writes to `.git/index`, `.git/HEAD`, and ref files on every operation, and a file-provider sync engine racing those writes produces phantom changes, dangling refs, and occasionally an unrepairable tree. A measurement in `sec-five-machine-storage` found 91 git repositories living inside the Dropbox tree on the author's machine, 23 under `prj` and 68 elsewhere, and calls moving them out the highest-value task in that entire plan. **(read)**

---

7. Security

The security posture is built in two movements: an operational credential architecture, and then a separate post that asks whether the operational architecture is actually secure. The distinction is stated explicitly: 'security controls and operational procedures are not the same thing'.

7.1 The three-tier secrets scheme

Tier 1, long-lived API keys: `pass`. The Unix password store, chosen because it is a roughly 700-line bash wrapper over GPG with a small audit surface, and because the rest of the construct is auditable to source and the secrets layer is the one where that matters most. Entries live in `~/.password-store/` and sync through a **private** git remote, with the corpus recommending a self-hosted Gitea or Forgejo instance over GitHub on the grounds that entry names and timestamps are themselves sensitive metadata even when the ciphertext is safe.

Tier 2, cloud credentials: AWS IAM Identity Center. Short- lived tokens via `aws sso login`, cached under `~/.aws/sso/cache/` rather than a long-lived `~/.aws/credentials`. The legacy file is moved aside, verified unnecessary, then rotated and deleted.

Tier 3, residual safety net: `gitleaks` at staging. Catches what the first two tiers missed: a

credential pasted into a notebook output cell, a private key copied in for testing, a `.env` that should have been ignored.

The injection pattern. The construct's convention is per-process injection, not environment export:

```
with-secret() {
  local var_name="$1"; local pass_path="$2"; shift 2
  local value
  value="$(pass "$pass_path")" || return 1
  env "$var_name=$value" "$@"
}
```

`export ANTHROPIC_API_KEY=$(pass anthropic/api-key)` is labelled WRONG in the source, because it is visible to every subprocess for the session's lifetime; `with-secret ANTHROPIC_API_KEY anthropic/api-key claude` is labelled RIGHT because the value reaches only the wrapped command's process tree. Tool-specific wrappers (`claude`, `nvim`) are thin functions over `with-secret`, using `command <tool>` to avoid recursion. **(read)**

7.2 The threat model and its controls

The threat model is explicitly scoped: a personal, not enterprise, multi-laptop cluster. In scope are physical theft, remote compromise, accidental exposure through a public commit or cloud sync, and credential reuse after a service breach. Out of scope, and named as such, are nation-state actors, supply-chain attacks on Homebrew itself, and firmware-level attacks.

Seven controls, ordered by the corpus's own priority:

1. **FileVault on every machine**, verified with `fdesetup status`. Stated as a prerequisite for every other control: a laptop with FileVault off and an encrypted `pass` store is said to be less secure than one with FileVault on and a plaintext password file, because `~/.gnupg/` is readable on the former. Screen lock at 60 seconds or less.
2. **GPG subkey architecture.** A certify-only master key moved to offline encrypted storage, a separate encryption subkey (ECC, Curve 25519, one-year expiry) for daily `pass` operations, and an optional signing subkey. The only reliable verification is `sec#` rather than `sec` in `gpg --list-secret-keys`. Ed25519 is the stated default for all new keys in 2026; RSA is not recommended.
3. **Per-machine SSH keys**, generated on each machine with a `$(hostname)-$(date +%Y%m)` comment and registered individually with every service. The rationale is selective revocation: retiring one machine removes one public key, where a shared key forces simultaneous rotation everywhere. The operational cost of registering each key on each service is acknowledged as the price of that property.
4. **Plaintext credential audit.** An explicit sweep for `~/.aws/credentials`, `~/.npmrc`, `~/.netrc`, `~/.config/rclone/rclone.conf`, and grep for token-shaped strings under `~/.config/`, with each finding migrated into `pass`.
5. **Clipboard security.** `pass -c` clears after 45 seconds, and any clipboard manager defeats that. iCloud Universal Clipboard is singled out as the most overlooked vector: it is on by default, it is labelled 'Handoff' rather than anything mentioning the clipboard, and it transmits `pass -c` output between machines within seconds.

6. **Dotfiles repository hardening.** Verified private, branch protection on `main` with `enforce_admins`, and periodic review of authorized keys via `gh api user/keys`.
7. **GPG agent cache TTL.** `default-cache-ttl 300`, `max-cache-ttl 600` in the threat-model post, with a note to reduce it further on machines used in public settings. (The secrets post gives 3600/28800; the tightening is a deliberate later revision, not an inconsistency.)

The quarterly review is treated as the most important control. Twelve checks, assembled into `analysis/configs/security-audit.sh` and put on a calendar. The stated principle: controls that are not periodically verified are controls in name only, and the common failures in a personal cluster are not sophisticated attacks but defaults left in place. **(read)**

Acknowledged residual gaps, stated by the corpus itself: macOS only, no hardware security key coverage, no SSH-server hardening, supply chain excluded, and `pass` itself having received no security updates since 2018 (with `passage` and `gopass` named as maintained successors worth evaluating).

---

8. Sync and storage

This is the part of the corpus with the most visible change of mind, and the sequence is worth preserving because the reversal is argued rather than quietly applied.

8.1 The three-layer framing

`sec-dropbox-to-portable-sync` argues that 'sync everything via one cloud provider' bundles three semantically distinct concerns:

- **Configuration distribution** wants version history, atomic deploys, and rollback. Git solves this; cloud sync only approximates it.
- **Project content sync** wants diff-awareness for large files, format-appropriate conflict semantics, and the ability to exclude derived artifacts.
- **Append-only history** wants either single-machine writes or a sync mechanism that understands append semantics. Whole-file cloud sync produces a conflict copy on every overlap. The concrete instance: eleven to twelve conflict copies of `~/.claude/history.jsonl` across five machines in a few months, each holding several hundred kilobytes of unique prompts, appearing with no notification and no error.

For append-only files the corpus offers three honest choices, in order of preference: do not sync them at all and accept per-machine history; designate one canonical machine; or adopt an append-aware tool (`atuin` is named as the example of the pattern done right).

A distinct and useful category is named here: **reconstructable local caches**. The worked case is a Maildir under mutt, mbsync and notmuch. The IMAP server is already the canonical source, so syncing the Maildir duplicates work IMAP performs, triples disk footprint, strains sync engines with hundreds of thousands of one-message files, and shares a Xapian B-tree across machines in a way that is not safe. The correct move is not to choose a sync mechanism but to rebuild from the authoritative source on each machine. The category generalizes to Homebrew caches, language-server indexes, browser caches, and `ccache`.

8.2 The migration discipline

The migration sequence is designed so that the backup pipeline remains valid at every intermediate step, and so that stopping part-way leaves a working system:

1. Confirm the dotfiles foundation with `install.sh --dry-run` on a clean account.
2. Introduce `PRJ_ROOT` in `.zshenv`, still resolving to the Dropbox path. Replace every literal path reference with the variable. Nothing changes visibly; the next move becomes a one-line edit.
3. Pilot a single small project at the new location.
4. Migrate the rest in batches, a batch a day.
5. Repoint backup sources to `$PRJ_ROOT`. The `launchd` plists do not change; only the scripts they call.
6. Resolve the append-only history question per file and record the choice in the dotfiles README.
7. Verify on a clean user account.
8. Optionally remove Dropbox, which most readers are told they do not need to do.

The two-pass indirection in step 2 is the generalizable technique: introduce the abstraction while it still resolves to the old value, then change what it resolves to.

8.3 The reversal: object storage over sync

`sec-object-store-research-data` and `sec-five-machine-storage` (both currently `draft: true`) revise the earlier recommendation. The earlier post recommended a hybrid of per-project git remotes for code and Syncthing or rsync for data. The revision states that the git half held up and the data half did not, because Syncthing and rsync both presume a mutable copy on every machine, which is the assumption underneath most of the catalogued failures. Choosing between them is characterized as choosing which flavor of the same problem to accept.

The replacement position: **data is written once under a versioned key, never edited in place, and fetched on demand**. The laptops hold caches, not replicas, and there is nothing to reconcile. The mechanism is three moves: a manifest in each repository, a `make data` pull step, and a `.gitignore` that ignores the bytes. A key plus a content hash becomes a durable citation.

The implementation tested is SeaweedFS, chosen over MinIO (whose open-source upstream is archived) and Garage (which implements neither object lock nor versioning). The corpus is careful about what the published comparisons optimize for, noting that they assume small memory footprints and high-latency links rather than one laptop on a LAN. Three test results are reported: object lock survives a restart, including refusal of deletion by version id; a `Read, List` identity is genuinely refused writes, overwrites, delete markers, and bucket creation (which matters because it means a client cannot even hide an object); and memory consumption of `weed server` is explicitly reported as **not measured**, flagged as the one result that could send the design back to a store that cannot enforce immutability.

8.4 The five-layer architecture at five machines

The current end state, for one Linux server and four workstations, with the design principle that **no layer uses a file-sync daemon**:

Layer	Mechanism
1. Network	Tailscale on all five machines; nothing port-forwarded
2. Configuration	Private GitHub `~/dotfiles` plus `install.sh`
3. Secrets	`pass`, GPG-backed, own private git remote
4. Code and authored text	One GitHub repo per project, cloned to `~/prj`
5. Data	SeaweedFS object store, manifest plus `rclone` pull

Network comes first because every other layer assumes the machines can find each other, and because it is the step with no downside if the rest of the plan is abandoned. Secrets are split out from configuration because the failure modes differ: a leaked `.zshrc` is embarrassing, a leaked credential is an incident. Layer 4 is defined by material rather than file extension: R, shell, LaTeX, Markdown and BibTeX are all 'code' because they are text a human edits, which is what git is good at. **(read)**

One deliberate, counterintuitive assignment: the oldest and slowest machine (a 2015 MacBook Air) is the server, and the fastest (an Arch workstation) is a client, on the reasoning that availability matters more than speed for this service and availability is a scheduling property rather than a hardware one.

---

9. Backup

The stated principle from the construct anchor is 'two tiers or it is not backup': GitHub holds source of truth and history, an external SSD holds the binary artifacts that should not be committed, and a single-tier backup is a single point of failure for both privacy and recovery.

The dedicated post implements four:

1. **Automated git commits and pushes every 15 minutes**, driven by a `launchd` job over all dirty repositories.
2. **Cloud synchronization** for continuous file-level replication, explicitly scoped: this tier exists to hold an off-site replica, not to be the canonical home of project content.
3. **Time Machine** to an external USB drive, with customized exclusions.
4. **Bulk GitHub archival**, a three-phase script (backup, verify, delete) that mirrors 400-plus private repositories with their GitHub-side metadata (issues, pull requests, releases, wikis) and supports selective deletion only after verification passes.

The opening claim is that GitHub alone is not a backup, because the source-of-truth tier must itself survive loss of the account or the platform. The archival leaf exists for that reason. **(heading, with extracts read)**

A recurring hardening detail: `rclone sync` exits 0 when the source directory does not exist, so every backup script carries an explicit `[[ -d "$PRJ_ROOT" ]]` guard and fails loudly on a missing source. Silent success is treated as the worse failure.

---

10. Reproducibility

10.1 The two-tier stack

The corpus is explicit that reproducibility is two tiers, not one. The **compendium** pins what an analysis depends on; the **construct** pins what the analyst's environment depends on. Neither subsumes the other, and the construct anchor names the gap that makes the second tier necessary: `renv.lock` does not pin the system R version, and nothing in a compendium pins the editor, shell, or multiplexer. Most of the time those are irrelevant; on the occasions they are not, the absence of a workflow-level record is what makes the failure hard to diagnose.

10.2 The five pillars

Every `zzcollab` workspace rests on five components:

1. **`Dockerfile`** defining the base image (`rocker/tidyverse` at a pinned R version), system dependencies, and the `analyst` user.
2. **`renv.lock`** pinning exact package versions.
3. **`.Rprofile`** activating renv, setting repository mirrors, and detecting whether the session is inside a container.
4. **Source code**, with analysis scripts in `analysis/scripts/` producing figures and derived data, and the narrative in `analysis/report/` loading pre-computed results rather than running the analysis inline. The separation is deliberate: the narrative renders fast and the analysis code is independently testable.
5. **Data**, with `raw_data/` immutable and `derived_data/` regenerable, each carrying a provenance README.

Two positions on containers are stated as principle: images are **built from a `Dockerfile`, never pulled from a personal registry**, because a registry image is opaque and a Dockerfile is auditable; and on Apple Silicon the build pins `--platform=linux/amd64` to keep results consistent with x86_64 collaborators and CI, accepting emulation overhead as the price. A related container gotcha is recorded: without `RENV_PATHS_CACHE` set inside the image, `renv::restore()` can install into the host library, producing the failure mode where the analysis succeeds locally and fails in CI with an identical lockfile. **(read)**

10.3 Testing

`tinytest` is the standard, with tests in `inst/tinytest/` named `test-*.R`. A dedicated post documents converting a package's suite from `testthat` to `tinytest` and reports losing roughly two-thirds of the lines without losing coverage, on the grounds that `testthat`'s ceremony is not free. A second case study (`ztable1`) records a seven-round migration arc, a Python translator for the mechanical part, and a helper-file consolidation problem that the naive approach does not survive.

Beyond unit tests, the taxonomy is four-part: unit tests, data validation tests (`assertr`, `stopifnot`), integration tests, and reproducibility tests (seed management, testing against known results, package version testing). The 55-item analysis initiation checklist puts seven data-integrity assertions (column count, column names, ID format, categorical levels, demographic coding, visit set, numeric plausible ranges) into `inst/tinytest/` before any analysis code is written. **(heading)**

10.4 Tiered CI

The CI position is the sharpest epistemological statement in the corpus. The finding that motivates it: after adding a single explicit failure check, five previously 'passing' projects turned honestly red. Three of five had `renv.lock` files pinning far fewer packages than their source actually used, meaning researchers were running in environments their lockfiles could not reproduce. The stated diagnosis is that a workflow which prints findings without erroring on them is indistinguishable, in GitHub's interface, from a workflow that found nothing wrong.

The response is a typology rather than a single template. Four workspace types (tool package, LaTeX manuscript compendium, Quarto compendium, Quarto blog post) get different combinations, because one template for all four meant blog posts forced through `R CMD check` while compendia had their render untested. Three tiers by verification purpose, each on its own cadence and each catching a different regression class.

Two named failure classes recur and are worth recording because they are not obvious: Posit Package Manager binary identifiers (the `-N` suffix) age out of the snapshot registry and break previously working lockfiles; and `setup-r@v2`'s `r-version: 'release'` drifts against a pinned lockfile, surfacing as opaque compilation errors against the wrong headers. The fix for the second is `r-version: 'renv'`, sourcing the R version from the lockfile.

The post also carries an in-place correction block reversing two of its own recommendations after multi-repo deployment: `setup-renv@v2` is incompatible with the zzcollab `.Rprofile` gate that suppresses renv activation outside a container (so the job installs nothing and passes), and `renv::status()$synchronized` is an unstable internal field that returns `FALSE` after a renv version downgrade regardless of actual state. **(read, in part)**

---

11. Authoring and publishing

11.1 Quarto as the document layer

Quarto is graded a **strong** recommendation and is intended to subsume R Markdown, Jupyter, and standalone LaTeX. Site-level defaults (`_metadata.yml`) set `freeze: auto`, `toc: true`, HTML with code folding and code tools both off, and PDF via `xelatex` with `DejaVu Sans Mono`, one-inch margins, and 11pt. The monofont choice is specifically for Unicode box-drawing characters in terminal transcripts.

Post discovery is by glob (`../posts/*/*/index.qmd`) with no site-level enumeration, so adding a post requires touching no shared file. A three-layer symlink architecture reconciles Quarto's expectation of `index.qmd` at the discovered root with the rrtools convention of `analysis/report/index.qmd`: root-level symlinks for `index.qmd`, `data`, `figures`, `media`, plus report-level symlinks pointing up one level, so relative paths resolve identically whether Quarto renders from the project root or from the report directory. A hard constraint was found by experiment and is recorded: **Quarto's listing glob does not follow symlinked directories**, which is why the project directories live physically inside the site tree rather than being symlinked in from `~/prj/blog/`. **(read)**

11.2 The post template

Every post is generated from one scaffold whose authoring instructions remain in the file as HTML comments. The scaffold enforces a fixed narrative shape:

Motivations (four to five bullets), Objectives (four numbered verifiable end states), What is X (one-sentence definition, one analogy, one concrete example), Prerequisites (OS version, hardware, prior knowledge, time estimate), the body, Verification (one-liners per layer), Uninstall / Rollback, Daily Workflow (a command table), Things to Watch Out For (five to seven gotchas, each with symptom and fix), Lessons Learned split into Conceptual / Technical / Gotcha-shaped, Limitations (four to six honest bullets), Opportunities for Improvement (five to six numbered), Wrapping Up, See Also, Reproducibility (a version matrix with a 'last verified' date), and Let's Connect.

Two of these sections carry more weight than their placement suggests. **Uninstall / Rollback** appears in nearly every setup post, in dependency order (containers and project data first because they hold work, then dotfiles, then plugin caches, then system packages), which makes each post falsifiable in a way a pure installation guide is not. **Reproducibility** with a dated version matrix is what converts a blog post into a citable snapshot of an environment.

The scaffold also carries a pre-publish QA checklist that gates `draft: false`, including four image slots with a `media/images/README.md` attributing provenance for each, and four prose-quality checks: zero emoji, zero em dashes, single quotes in prose, and 78-character line wrap throughout. Drafts remain `draft: true` until every box is ticked, and several posts in the corpus are currently held there.

11.3 House style

The prose conventions are consistent enough across sixty-one posts to be stated as rules: academic and scholarly register; no emoji; no em dashes; single quotes; US English; 78-character wrap; measured rather than promotional language; first person reserved for the 'I did not really know how X until Y' hook that opens roughly a third of the posts, with the body shifting to 'we' or to the impersonal. Figures carry descriptive captions written to describe the actual image. Cross-references between posts are dense and directional, and each post ends with a 'Related posts in this cluster' block giving a recommended reading order.

11.4 Review methodology

Four parallel posts define review frameworks for AI-assisted blog posts, textbook chapters, research papers, and R packages. All four share a five-phase structure (structural review, unit-by-unit review, verification, integration review, and a safety review) plus a per-unit annotation template with Status, Understanding, Concerns, Questions, and Changes Required.

The common element is an **explicit ownership test** rather than a sense that the review went well. The stated distinction: 'I reviewed it' and 'I can rebuild this from memory' are different claims, and only the second licenses shipping under one's own name. The reasoning is that an author who cannot explain an implementation cannot debug it, extend it, or defend it to a reviewer, and that when the same assistant writes both the implementation and the tests, passing tests carry far less information than they appear to. Coverage is called a floor, not a review.

The code-review post also states an R-ecosystem limitation plainly rather than papering over it: there is no `npm audit` equivalent, `oysteR` queries the OSS Index and `riskmetric` scores maintenance signals, and neither substitutes for reading the dependency list and knowing what each entry is for. Secret scanning is directed at git history as well as the working tree, because a secret committed and later removed is still in the history. **(read, in part)**

---

12. AI-augmented work

The corpus admits generative AI on a specific and narrow contract.

The primary integration is a shell CLI, not an editor. Claude Code invoked from inside `~/prj` is the recommended form, graded above in-buffer plugins, above graphical AI editors, and above browser-based prompting. The reasoning is architectural rather than about capability: the shell CLI is the only form in which every existing piece of construct machinery (modal editor, secret-scanning git wizard, container-bound R session, reproducible compendium) operates unchanged, and in which the assistant's diffs pass through `zzgit` like any other diff. A graphical editor that applies diffs outside the git wizard produces an unscanned commit; a browser chat surface makes the transcript a load-bearing artifact that lives outside git, outside the dotfiles repository, and outside the scanner's reach.

The human owns the buffer and the staged commit. The recommended default is the diff loop: every change to versioned source passes human review before commit. Autonomous loops are admitted for narrow, well-scoped, bounded tasks, but only as an explicit opt-in.

`CLAUDE.md` is the load-bearing artifact. A short per-project file at the git root documenting Conventions, Layout, Persona, and Don'ts, committed alongside the code, costing roughly fifteen minutes to author and amortized over every subsequent invocation. The instruction on writing it is explicit: intentionally short, specific over general. A file that says 'be careful' is useless; one that says 'do not modify `data/raw/`' is operative. A global `~/.claude/CLAUDE.md` carries the cross-project preferences (R style, citation conventions, scholarly voice, no em dashes).

The API key is never exported. Consistent with the secrets tier, `claude` and `nvim` are wrapped in `with-secret` functions that inject the key for the duration of one command, and long-lived keys in `.zshrc` are named as a specific anti-pattern.

A failure catalog exists and is used. Hallucinated APIs in less-common R packages, stale context, secret exposure, cost runaway, and the model's tendency to over-engineer beyond the request. The review methodology posts turn the same concerns into procedure.

Adjacent AI-touching workflows in the corpus follow the same custody rule. The voice copyedit pipeline (Whisper for transcription, a delimiter-based anchor grammar, an explicit 'apply-only contract' with absolute constraints and a do-not-touch list, then resolve, verify, apply, verify, stage selectively, commit) never lets the transcription write directly. The `tn` daily research log talks to the Gemini API from a single bash script that stores one note file per project. In each case the model produces a proposal and a versioned, human-reviewed step applies it. **(read for the editing post; heading for the other two)**

---

13. The style, stated as invariants

Reading across the whole corpus, twelve rules recur often enough and are argued consistently enough to be called the style:

1. **Name layers, not tools.** Any tool is replaceable; the layer is not, and the convention outlives both.
2. **Configuration is source code.** It lives in git, it is installed by a tested script, it is dry-run before it is run, and it is verified on a second machine.

3. **Cloud sync is not version control, and git must never live inside it.** This is the single most repeated rule in the corpus and the one with a measured incident behind it.
4. **Separate by semantics before choosing a mechanism.** Configuration, secrets, code, data, append-only history, and reconstructable caches each want a different answer; lumping them enforces no useful constraint.
5. **Prefer data that is written once and fetched, over files that are mutated everywhere and reconciled.** Laptops are caches, not replicas.
6. **A helper is a function only if it must mutate the calling shell.** Everything else is a versioned, linted, hardened script in `~/bin`, reachable by cron, Make, Docker, and CI.
7. **Secrets are state.** They get an inventory, a backup, a rotation schedule, a recovery procedure, and per-process injection rather than environment export. Three tiers: encrypted store, short-lived cloud tokens, staging-time scanner.
8. **Prefer selective revocation to convenience.** Per-machine SSH keys, GPG subkeys with the master offline, and expiry dates on everything, accepting the registration cost.
9. **Verify periodically or the control does not exist.** A quarterly 30-minute audit is worth more than any individual control.
10. **Make CI capable of failing.** A check that prints findings without erroring is worse than no check, because it manufactures confidence.
11. **Publish the limitation.** Every post carries Limitations, Things to Watch Out For with symptom and fix, and a dated version matrix; several carry in-place corrections reversing their own earlier recommendations, and one reports that a layer claimed complete is not complete on the author's own machine.
12. **The human owns the artifact.** Assistants propose diffs; the human reviews, stages through the scanner, and commits. Ownership is defined as the ability to rebuild the work from memory, not as having read it.

---

14. Internal tensions and open items

Stated here because a reference that only records the resolved parts would misrepresent the corpus.

- **Dropbox is still load-bearing.** The corpus argues at length for leaving it, and the working tree of this very repository is under `~/Library/CloudStorage/Dropbox/`. Layer 1 was designed, not finished, on the primary Mac; Layers 2 and 3 remain in progress; 91 git repositories were measured inside the sync tree. The migration sequence exists and is robust to interruption, and it has been interrupted.
- **The object-store design is untested in one decisive respect.** `weed server` memory consumption on a 4 GB 2015 MacBook Air is explicitly unmeasured, and the corpus names it as the result that could invalidate the plan. Both object-storage posts are still `draft: true`.
- **GPG agent TTL differs between two posts** (3600/28800 in the secrets post, 300/600 in the threat-model post). The tightening is deliberate, but no single document states which is current.
- **`pass` has had no security updates since 2018**, and the corpus says so while continuing to recommend it, naming `passage` and `gopass` as successors to evaluate.
- **The construct is macOS-centric in its security layer and single-user throughout.** Linux equivalents for FileVault, agent caching, and clipboard control are acknowledged as existing but unwritten; team-scale conventions are declared out of scope.
- **Several posts carry unfilled placeholders.** The construct anchor still ships

`github.com/mygit` and `user@example.com` in its Feedback section and remains flagged in its own QA checklist.

---

15. What the corpus does not cover

For completeness, and because the construct anchor says so directly: full-disk-encryption policy beyond enabling FileVault, firmware passwords, secure boot, hardware security keys, SSH-server hardening, supply-chain verification, off-site cold backup beyond the GitHub archival leaf, HPC and institutional cloud substitutions for the AWS row, Windows anything, and any team-scale governance. The corpus positions these as layered beneath or beside the construct rather than inside it, and recommends following vendor or institutional guidance for each.

Appendix: post index by cluster

workflow-construct (7): `wf-construct-overview-anchor`, `wf-unix-workspace-config`, `wf-multi-laptop-dotfiles-bootstrap`, `wf-git-for-data-science`, `wf-neovim-data-science-ide`, `wf-modern-cli-tools`, `wf-linux-mint-on-macbook`.

workflow-extensions (3): `wf-claude-code-in-shell`, `wf-pocket-terminal-ttyd-tailscale`, `wf-voice-copyedit-pipeline`.

security-and-sync (3): `sec-three-tier-backup-architecture`, `sec-pass-password-manager`, `sec-aws-and-pass-secrets`, with `sec-multi-laptop-threat-model` as the cluster's fifth-ordered capstone.

cloud-sync-migration (3): `sec-dropbox-to-portable-sync`, `sec-object-store-research-data`, `sec-five-machine-storage`.

shell-and-git (2): `sh-scripts-vs-functions`, `sh-daily-research-log`.

zzcollab-compendia (9): `zc-quarto-compendium-intro`, `zc-blog-post-template`, `zc-markdown-to-blog-workflow`, `zc-analysis-initiation-checklist`, `zc-analysis-profile-walkthrough`, `zc-manuscript-report-elements`, `zc-tiered-ci-strategy`, `zc-github-actions-workflows`, `zc-share-rmd-via-docker`, `zc-new-student-reading-guide`.

r-packages (7): `rp-code-review-methodology`, `rp-code-review-exemplar`, `rp-blog-review-methodology`, `rp-book-review-methodology`, `rp-paper-review-methodology`, `rp-package-update-workflow`, `rp-testing-data-analysis`, `rp-testthat-to-tinytest`, `rp-vim-r-repl-plugin`, `rp-zzvim-r-introduction`.

publishing (7): `pub-multi-language-quarto`, `pub-r-script-to-rmd`, `pub-statistical-computing-textbook`, `pub-open-textbook-guidebook`, `pub-hosting-quarto-books-netlify`, `pub-obs-r-screencasts`, `pub-ai-course-policy-survey`.

r-language (2): `rl-pipe-equivalence-myth`, `rl-dynamic-column-names`.

clinical-and-cloud (4): `cln-multilang-trial-validation`, `cln-aws-ec2-provisioning`, `cln-zzedc-investigator-independence`, `cln-shiny-docker-caddy-hosting`.

penguins-arc (8) and **shiny** (2) are the worked-analysis and application clusters; they exercise the style rather than defining it.

(Cluster counts are the declared `cluster:` field values; appendix listings include a small number of posts whose declared cluster differs from the thematic grouping used here.)

Rendered on 2026-09-12 at 11:46 PDT. Source: ~/prj/ryyblog/docs/rgtlab-style-white-paper.md